

Knowledge Graph Template Proposal

Two-Layer Template + Structured Rules + Ops-friendly Input Unit

1 Goal

We want a knowledge graph (or KG-backed knowledge base) that supports an agentic BI decision engine by enabling:

- **Constraint-aware filtering** (avoid infeasible / risky actions)
- **High-quality retrieval** (right knowledge for the right context)
- **Evidence-grounded recommendations** (provenance + testability)
- **Scalable ops experience capture** (ops can contribute without thinking in triples)

2 Two-Layer Template

2.1 Layer 1 — Graph Schema (Stable “World Model”)

Purpose: define core entities and relationships per vertical/business model so reasoning and retrieval can leverage domain structure (and metric semantics don’t get mixed across business models).

MVP Entities (Nodes):

- Merchant, Store/Channel
- SKU/Product, Category, Brand
- Campaign
- Policy, ActionType
- FunnelStage

MVP Relations (Edges):

- $\text{SKU} \rightarrow \text{belongs_to} \rightarrow \text{Category}$
- $\text{SKU} \rightarrow \text{of_brand} \rightarrow \text{Brand}$
- $\text{Campaign} \rightarrow \text{promotes} \rightarrow \text{SKU}$
- $\text{SKU} \leftrightarrow \text{substitutes_with} / \text{complements_with} / \text{bundled_with}$
- $\text{Policy} \rightarrow \text{applies_to} \rightarrow (\text{vertical} / \text{channel} / \text{category} / \text{SKU})$
- $\text{Policy} \rightarrow \text{constrains} \rightarrow \text{ActionType}$

Deliverable: a minimal “Funnel & Relationship Map” per vertical scenario that can evolve over time.

2.2 Layer 2 — Ops Knowledge Entries (Scalable “Experience Units”)

Purpose: capture operational experience as reusable, retrievable decision intelligence. Each entry should be a self-contained unit that can:

- match a merchant context + problem pattern
- propose checks/actions with clear preconditions
- state constraints, risks, expected outcomes
- provide provenance for trust and versioning

Key principle: Ops fills entries in a structured form; engineering maps entries to graph nodes/edges + indexed text for retrieval.

3 Ops-friendly “KG Input Unit” (Entry Schema)

Each entry is one “experience capture unit”. Recommended fields:

3.1 A) Context (Avoid Generic Advice)

- Vertical / Business model
- Merchant maturity (new vs mature store)
- Channel(s) (Shopify/Amazon/Meta/Google...)
- Region/market (if relevant)
- Seasonality / timeframe (if relevant)

3.2 B) Problem Pattern (What We’re Diagnosing)

- Funnel stage (acquisition / conversion / retention / inventory health)
- Symptoms (e.g., CVR drop, ROAS decline, high cart abandonment)
- Trigger signals (metrics + thresholds + time window)
- How ops recognized it (notes)

3.3 C) “If observed → check → because” (Structured Diagnostic Logic)

- What to check next (data/metrics/segments)
- Why it matters (mechanism / explanation)
- Expected outcomes of the check (what confirms/refutes the hypothesis)

3.4 D) Actions / Playbook (What to Do)

- Recommended action types (discount / price change / budget shift / pause SKU / creative refresh...)
- Preconditions (when it's safe/valid)
- Step-by-step operational steps
- Cost/effort level (low/med/high)
- Required data / API requirements

3.5 E) Outcome & Evidence (What Happened / What We Expect)

- Expected lift (which KPIs, direction, approximate magnitude)
- Time lag (when you should see impact)
- Side effects (e.g., margin compression, brand dilution)
- Confidence level (low/med/high) + evidence quality

3.6 F) Constraints & Failure Cases (How it Can Go Wrong)

- Hard constraints (must-not-violate)
- “Do NOT apply if ...” conditions
- Edge cases / failure stories (“didn’t work when ...”)

3.7 G) Provenance (Trust & Governance)

- Owner (ops/finance/marketing)
- Source link (doc, ticket, meeting notes)
- Version + last updated
- Approval status (draft / reviewed / approved)

4 Structured Rule Schema for “If observed → check → because”

To make reasoning **controllable and testable**, we store diagnostic logic as structured rules (not just free text).

4.1 Rule Object (MVP)

- **id**
- **scope**: vertical, business model, channel, category (optional)
- **trigger**: observable condition(s)
- **checks**: list of what to check next + why
- **because**: rationale/mechanism (grounding)
- **suggested_actions** (optional): candidates gated by preconditions
- **provenance**: owner/version/source link

4.2 Example (Pseudo-JSON)

```
{
  "rule_id": "rule:cvr_drop_checkout_v1",
  "scope": {
    "vertical": ["beauty", "apparel"],
    "channels": ["shopify"]
  },
  "trigger": {
    "all": [
      {"metric": "checkout_conversion_rate_7d", "op": "<=", "value": 0.85,
        "relative_to": "28d_baseline"},
      {"metric": "traffic_7d", "op": ">=", "value": 0.95, "relative_to":
        "28d_baseline"}
    ]
  },
  "checks": [
    {"check": "segment_by_device", "why": "mobile UX changes often affect
      checkout"},
    {"check": "payment_method_failure_rate", "why": "payment issues can
      reduce checkout CVR"},
    {"check": "shipping_cost_change", "why": "unexpected shipping increases
      drive abandonment"}
  ],
  "because": "CVR drop with stable traffic suggests checkout friction
    rather than demand shift.",
  "suggested_actions": [
    {"action_type": "FIX_CHECKOUT_SETTINGS", "preconditions": ["
      payment_issue_confirmed"]},
    {"action_type": "PROMO_FREE_SHIPPING", "preconditions": ["
      shipping_cost_increase_confirmed"]}
  ],
  "provenance": {
    "owner": "ops",
    "version": "1.0",
    "source_link": "..."
  }
}
```